

Phase 6 - limitations

- [PHASE 6 — LIMITATIONS](#)

- [1. Financing terms have no source in Phase 1 data](#)
- [2. purchase_price defaults to asking_price , which is a listing price, not a confirmed sale](#)
- [3. Locality appreciation is unreliable for ~14% of the dataset \(inherited from Phase 3\)](#)
- [4. Only 44.5% of properties have both expense and rental history](#)
- [5. Vacancy defaults to 0.0 \(best case\) when no data exists, not a “typical” estimate](#)
- [6. Scenario shocks \(Upside/Downside\) are documented assumptions, not calibrated](#)
- [7. DSCR, payback period, and IRR can legitimately be None](#)
- [8. No live PostgreSQL was available to run the DB-default path in this environment](#)
- [9. Batch runtime at full portfolio scale \(8,000 properties\) is not measured](#)
- [10. Post-delivery audit fix: roi_total_holding_period was overstated \(now fixed\)](#)

PHASE 6 — LIMITATIONS

Consolidated from the phase report. Kept as a separate document per Master Prompt Section 16 (Documentation must cover “Limitations” as a first-class topic).

1. Financing terms have no source in Phase 1 data

`core.properties` has no loan/interest-rate/LTV columns. `FinancingTerms` defaults to `down_payment_pct=1.0` (all-cash) precisely because there is no historical financing data to derive a “typical” default from — inventing one (e.g. “most buyers put 20% down at 9%”) would be exactly the kind of silent guess Section 3.2 prohibits. A leveraged analysis requires the caller to state real loan terms.

2. `purchase_price` defaults to `asking_price` , which is a listing price, not a confirmed sale

Where `core.transactions` contains a historical sale for the same `property_id` , it is surfaced in `data_loader` ’s output (`observed_transaction_history`) but is **not** automatically substituted for `asking_price` . Asking price and realized sale price are different facts (asking price is what a seller is asking *now*; a past transaction is what actually happened *then*, potentially for a different owner/condition). Silently picking one over the other would blur that distinction; the engine leaves the choice explicit.

3. Locality appreciation is unreliable for ~14% of the dataset (inherited from Phase 3)

Checked directly against `core.market_monthly` for all 90 localities: median historical CAGR is a plausible 12.1%/yr, but 13 localities (14.4%) exceed 30%/yr, up to 132.75%/yr for one locality. This lines up with the Phase 3 EDA report’s documented locality-price-generator compounding bug. `data_loader.locality_appreciation()` flags anything above a documented 20%/yr sanity ceiling as `OBSERVED_BUT_UNRELIABLE` , and `engine.resolve_inputs()` refuses to use a flagged value as a default `appreciation_rate` — the field becomes `INSUFFICIENT DATA` instead, unless the caller supplies an explicit override. The 20%/yr ceiling itself is a documented domain judgement call (very strong, sustained multi-year residential CAGRs above that are rare in practice), not a statistical test derived from this data.

4. Only 44.5% of properties have both expense and rental history

`core.expenses` covers 62.1% of the 8,000 properties, `core.rentals` covers 69.7%, and the two together (this phase's demo-batch eligibility bar) cover 44.5% (3,564 properties). For the remaining 55.5%, NOI/yield/cap-rate cannot be fully resolved from OBSERVED data alone — the engine correctly returns `INSUFFICIENT DATA` for those unless a caller supplies `annual_operating_expenses` and/or `vacancy_rate` explicitly. This is a real data-coverage gap, not an engine defect, and it is exactly the kind of fact Phase 9's `financial_completeness` decision-confidence input (§9.6) needs.

5. Vacancy defaults to 0.0 (best case) when no data exists, not a “typical” estimate

If neither the caller nor the property's own rental history supplies a vacancy rate, the engine sets `vacancy_rate = 0.0` — this is explicitly labeled `ASSUMED_DEFAULT` and added to `insufficient_data_fields`, and the phase report calls out that this makes yield/cash-flow figures an **upper bound**, not a realistic estimate. An alternative design (defaulting to a portfolio-average vacancy rate) was considered and rejected: a portfolio average is itself a kind of invented number for a specific property that has no observed vacancy history, and would be harder for a downstream reader to recognize as an assumption than an explicit 0.0 flagged as best-case.

6. Scenario shocks (Upside/Downside) are documented assumptions, not calibrated

`scenarios.SCENARIO_SHOCKS` (e.g. Downside: appreciation –5pp, vacancy ×2, opex ×1.10) are stated stress magnitudes chosen for directional plausibility, not fitted to any historical volatility measure in this dataset (none exists at this granularity). They must not be read as calibrated probabilities of anything — that distinction belongs to Phase 7's Monte Carlo engine (§7.7's UNCALIBRATED status), which this phase's `engine.py` is designed to be reused by unchanged.

7. DSCR, payback period, and IRR can legitimately be `None`

- `dscr` is `None` for any all-cash deal (no debt service to divide by) — this is correct behavior, not a missing value.
- `payback_period_years` is `None` when the cash-flow schedule never turns cumulatively positive within the holding period (`test_payback_period_none_when_never_recovered`) — a real possible outcome for a low-yield property, not a computation failure.
- `irr` is `None` when the cash-flow series has no sign change (e.g. all-positive flows, or the bisection search over [-99%, 1000%] finds none) — mathematically, IRR is undefined in that case, and returning `None` is more honest than returning a number from a search that didn't actually converge.

8. No live PostgreSQL was available to run the DB-default path in this environment

Same situation as Phases 3 and 5: this sandbox has no PostgreSQL instance. All three orchestrators (`export_results_csv.py` , `generate_charts.py`) default to `--source db` (the production path, identical query pattern to Phase 5's `db/connection.py` usage) but were actually run here with `--source csv` , which reads the identical rows from `data/processed/*.csv` (the same files Phase 1 loaded into `core.*`). Recommendation: re-run with `--source db` against the user's local database to confirm the DB path end-to-end, not just its logic.

9. Batch runtime at full portfolio scale (8,000 properties) is not measured

The demo batch (n=25, 750 total engine calls across scenarios + sensitivity) runs in well under a second. No claim is made about runtime or memory behavior if this pipeline were run across the full 8,000-property portfolio; per Master Prompt §15, that should be measured before optimizing, not assumed.

10. Post-delivery audit fix: `roi_total_holding_period` was overstated (now fixed)

An independent review found `roi_total_holding_period` was computing `total_return / equity_invested` (a gross cash multiple) instead of `(total_return - equity_invested) / equity_invested` (a net return). A hand-checkable single-period case (invest ₹1,000, receive ₹1,100 – a 10% return, matching IRR exactly) was returning **110%** instead of the correct **10%**: overstated by exactly one full unit of equity (100 percentage points) in every case, because the formula never subtracted the original investment back out. This was not caught by the original test suite (no test directly checked this metric against a known value). **Fixed** in `engine.py::compute_metrics`, with two new regression tests (`test_roi_is_net_Return_Not_Gross_Cash_Multiple`, `test_roi_matches_irr_for_single_period_deal`) added to `test_phase6.py` – suite is now 25/25 passing. `docs/financial_results_csv/03_financial_metrics.csv` and `04_scenario_comparison.csv` were regenerated with corrected values. No other metric was affected by this bug or its fix.